PART A

Experiment 4

A.1 Aim: Implementing Git Flow and Collaborative Workflows using GitHub.

A.2 Prerequisite.

Windows PC, Git installed, basic command prompt knowledge.

A.3 Outcome

Students can initialize a Git repository, create commits, work with branches, merge changes, and resolve merge conflicts.

A.4 Theory

1. Git Flow

Git Flow is a branching strategy used to organize software development. It separates stable code from ongoing development and individual feature work. Different branches are used for different purposes. • **Main branch:** Contains the stable version of the project.

- **Develop branch:** Contains the latest combined development work.
- **Feature branches:** Used by individual developers to develop new features without directly changing the main branch.
- **Release branch:** Used to prepare a version for release.
- **Hotfix branch:** Used to fix urgent problems in the stable version.

For beginners, a simplified workflow using the main branch, feature branches, and Pull Requests is sufficient. **2. Collaborative Workflow Using GitHub**

A collaborative workflow allows multiple students or developers to work on the same project using Git and GitHub. Each person can work on a separate feature branch and upload their changes to GitHub without directly modifying the main branch.

The team leader can review the changes through a Pull Request before merging them into the main branch. **3. Important Git Concepts**

- **Repository:** Stores project files and their version history.
- **Branch:** An independent line of development used to work on a feature.
- **Commit:** A saved checkpoint of changes in the repository.
- **Push:** Uploads local commits to a remote repository such as GitHub.
- **Pull:** Downloads and integrates changes from a remote repository into the current local branch. • **Pull Request (PR):** A request to review and merge changes from one branch into another. • **Code Review:** The process of checking code quality and correctness before merging.

4. Advantages of Collaborative Workflow

- Multiple students can work on different features simultaneously.
- Changes can be tracked through commits.
- Pull Requests allow code review before merging.
- Working on feature branches reduces direct changes to stable code.
- GitHub helps team members share and manage project updates.

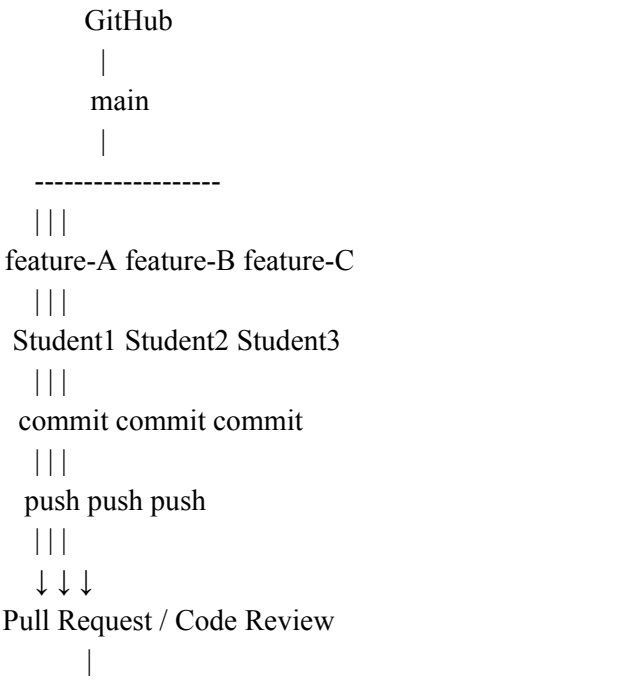
Procedures

Step 1 — Create GitHub repository

One student acts as Team Leader.

On GitHub:
New Repository → college-event-project
Keep it private/public as required.
Add the other students as Collaborators.
Step 2 — Every student clones the repository
git clone <repository-url>
cd college-event-project
Then check:
git branch
Initially:
main
Step 3 — Create a feature branch
Suppose Student 1 is working on Login.
git checkout -b feature-login
or the newer command:
git switch -c feature-login
Now:
main
  |
  └── feature-login
Explain:
"You are not working directly on the main project. You are working on your own copy/branch." Step 4 — Create/modify files
For example:
login.html
login.css
Then check what changed:
git status
Step 5 — Add changes
git add .
Then commit:
git commit -m "Added login page"
Explain:
Commit = checkpoint/save point.
Step 6 — Push branch to GitHub
git push -u origin feature-login
Now the branch appears on GitHub.
3. Pull Request — Most important part
On GitHub, students will see:
Compare & pull request
Student creates:
feature-login → main
Then writes:
Title:
Added login page
Description:
Implemented login form and basic validation.

Click:
Create Pull Request
Now the team leader reviews the code.
4. Review and Merge
Team leader checks:
Is the code working?
Is the code properly written?
Does it affect other functionality?
Are files unnecessary?
If everything is correct:
Merge Pull Request → Confirm Merge
Now:
feature-login
↓
Pull Request
↓
Review
↓
MERGE
↓
main
5. Very important: Before starting new work
Students should always update their local
main. git checkout main
git pull origin main
Then create a new feature branch:
git checkout -b feature-registration
This prevents students from working with an outdated
version. 6. Show the complete collaborative workflow
I recommend putting this on the board:



Merge
|
main
Commands students should remember
git clone <url>

git checkout main
git pull origin main

git checkout -b feature-login

git status
git add .
git commit -m "Added login page"

git push -u origin feature-login
After the Pull Request is merged:
git checkout main
git pull origin main
7. How to explain Git Flow
For beginners, don't start with git-flow commands. First explain the branching idea. A simplified Git Flow is:
main
↑
| Release
develop
↑
|
feature branches
Meaning
main → Stable/production version
develop → Latest combined development work
feature branches → Individual student work
For your students' practical:
main
↑
develop
↑

| | |
feature-1 feature-2 feature-3
However, for a beginner classroom practical, I recommend starting with: main + feature branches + Pull Request
Then introduce develop/release/hotfix branches after students understand the basic workflow. 8. Student practical activity
Divide students into groups of 4.
Project

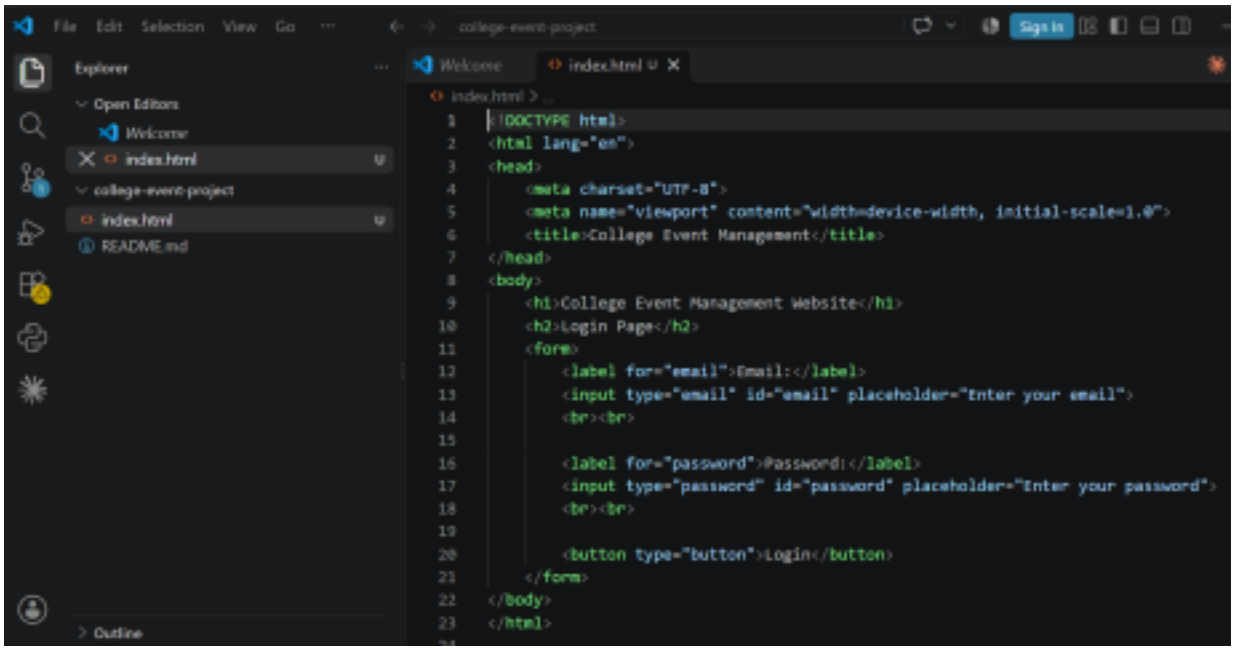
"College Event Management Website"
Student
Responsibility
Branch
Student 1
Login
feature-login
Student 2
Registration
feature-registration
Student 3
Events
feature-events
Student 4
Contact
feature-contact
Each student must:
Clone repository
Create feature branch
Create/modify files
Commit changes
Push branch
Create Pull Request
Team leader reviews
Merge PR
Pull latest main

PART B
(PART B: TO BE COMPLETED BY STUDENTS)

Roll No. A27	Name: Prajwal Ashok Zolage
Class: BE–AI&DS	Batch: A1
Date of Experiment: 18/08/2026	Date of Submission:18/08/2026

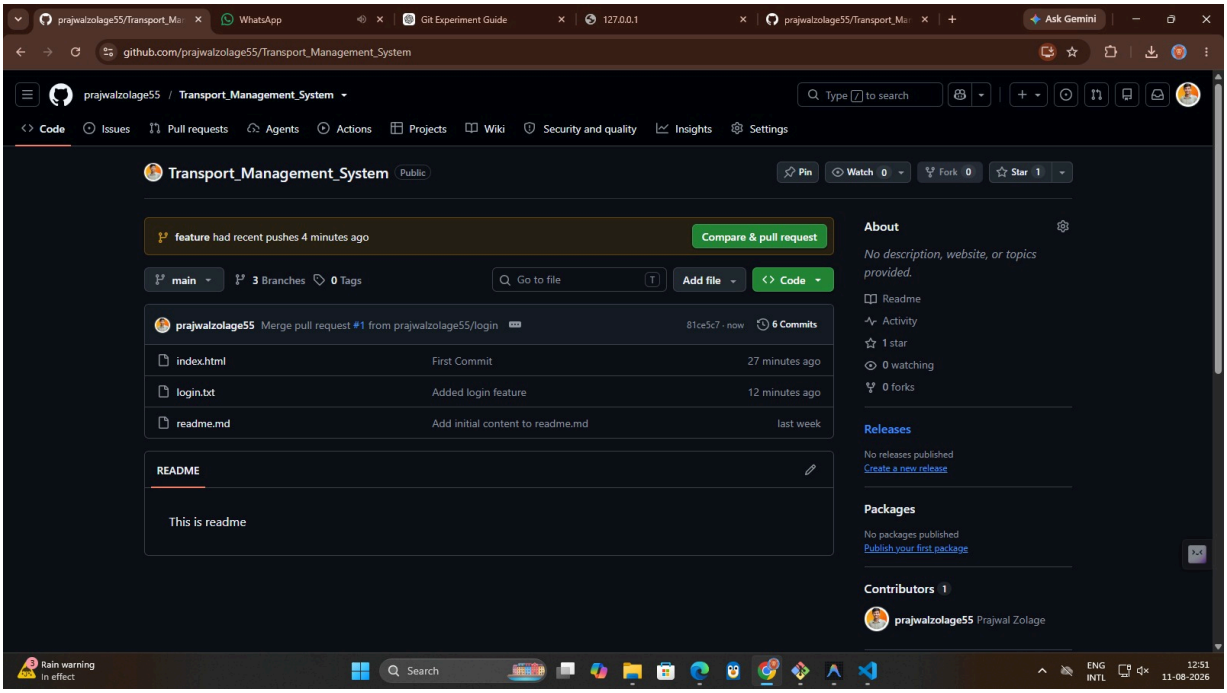
B.1 Code Screenshot:

- **HTML Code for the Login Page**



B.2 Input and Output:

- **Creation of GitHub Repository**



- **Cloning the Repository into Visual Studio Code**

```
PS C:\Users\Prajwal\OneDrive\Desktop\Data Analyst Agent\Transport_Management_System> git clone https://github.com/prajwalzolage55/
Transport_Management_System.git
Cloning into 'Transport_Management_System'...
remote: Enumerating objects: 148, done.
remote: Counting objects: 100% (148/148), done.
remote: Compressing objects: 100% (102/102), done.
remote: Total 148 (delta 56), reused 128 (delta 46), pack-reused 0 (from 0)
Receiving objects: 100% (148/148), 2.45 MiB | 3.12 MiB/s, done.
Resolving deltas: 100% (56/56), done.
PS C:\Users\Prajwal\OneDrive\Desktop\Data Analyst Agent\Transport_Management_System>
```

• **Checking the Current Git Branch**

```
Output  Problems  Debug Console  Terminal  Ports

PS C:\Users\Admin\OneDrive\Desktop\Transport_Management_System> git branch
  * main
PS C:\Users\Admin\OneDrive\Desktop\Transport_Management_System>
```

• **Creating the feature-login Branch**

```
Output  Problems  Debug Console  Terminal  Ports  powershell

PS C:\Users\Admin\Transport_Management_System> git branch
  * main
PS C:\Users\Admin\Transport_Management_System> git switch -c feature-login
  Switched to a new branch 'feature-login'
PS C:\Users\Admin\Transport_Management_System> git branch
  * feature-login
    main
PS C:\Users\Admin\Transport_Management_System>
```

• **Staging and Committing the Login Page**

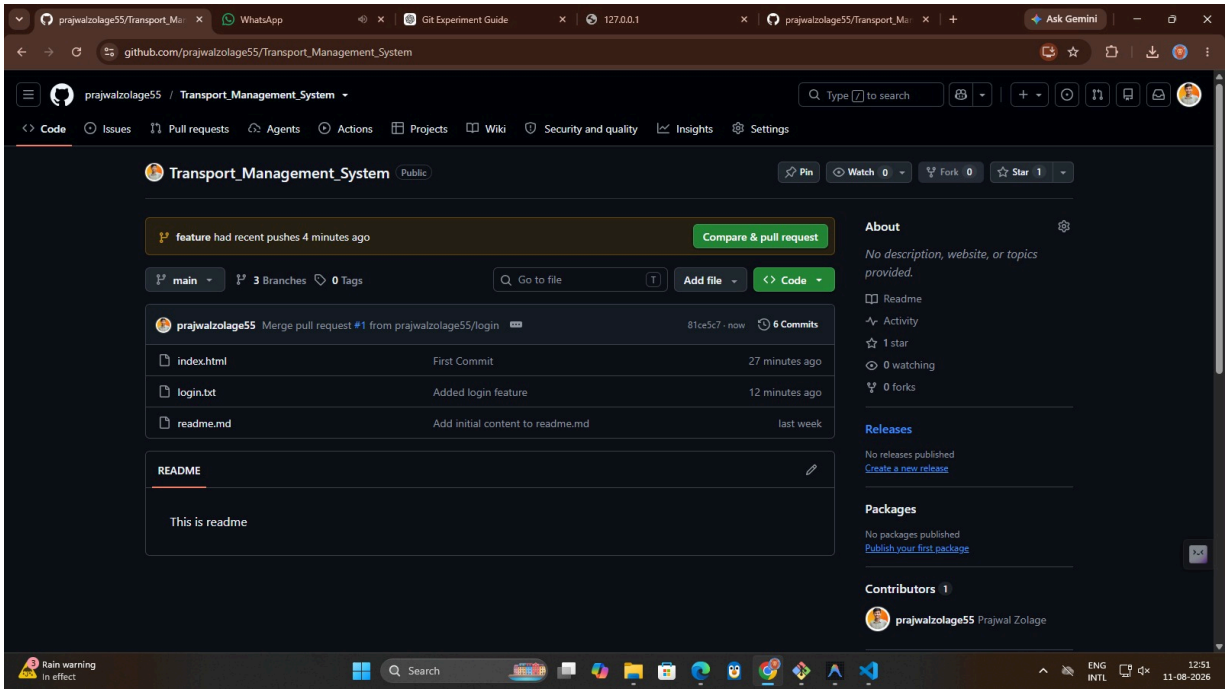
```
Output  Problems  Debug Console  Terminal  Ports  powershell

PS C:\Users\Admin\Transport_Management_System> git status
  On branch feature-login
  Untracked files:
    (use "git add <file>..." to include in what will be committed)
          index.html

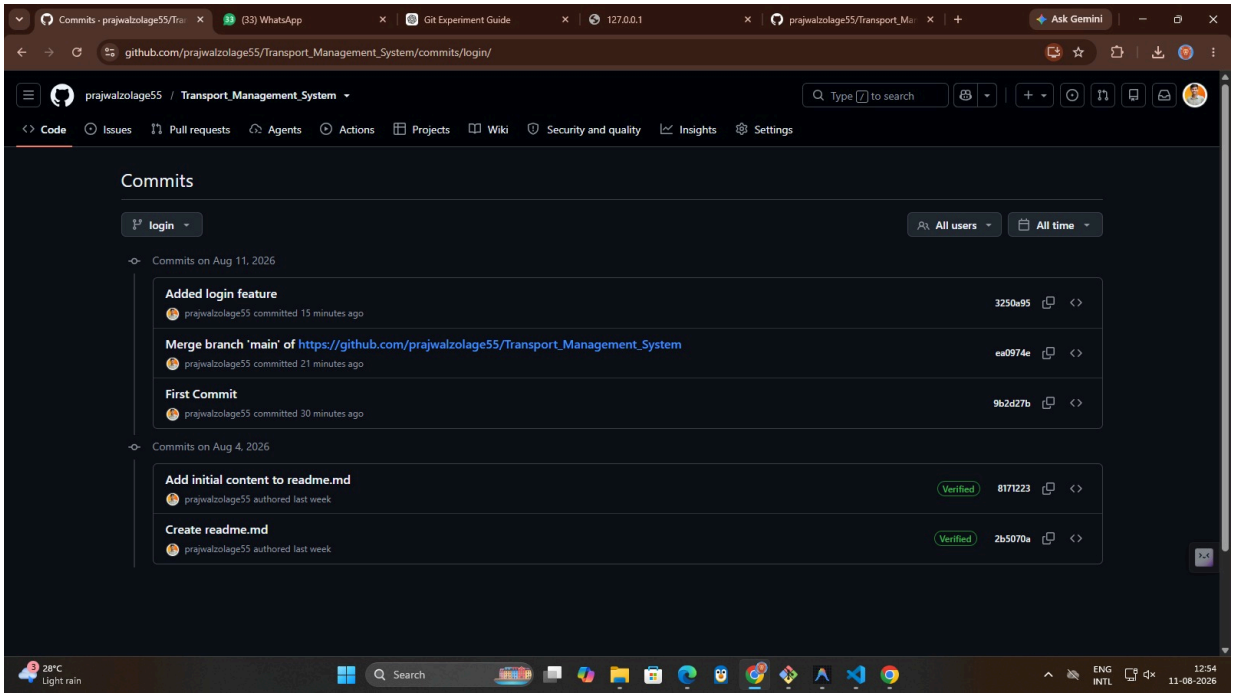
  nothing added to commit but untracked files present (use "git add" to track)
PS C:\Users\Admin\Transport_Management_System> git add .
PS C:\Users\Admin\Transport_Management_System> git status
  On branch feature-login
  Changes to be committed:
    (use "git restore --staged <file>..." to unstage)
          new file:   index.html

PS C:\Users\Admin\Transport_Management_System> git commit -m "Added login page"
  [feature-login 6d2d5f2] Added login page
   1 file changed, 23 insertions(+)
   create mode 100644 index.html
PS C:\Users\Admin\Transport_Management_System>
```

• **Pushing the Feature Branch to GitHub**



Successful Pull Request Merge



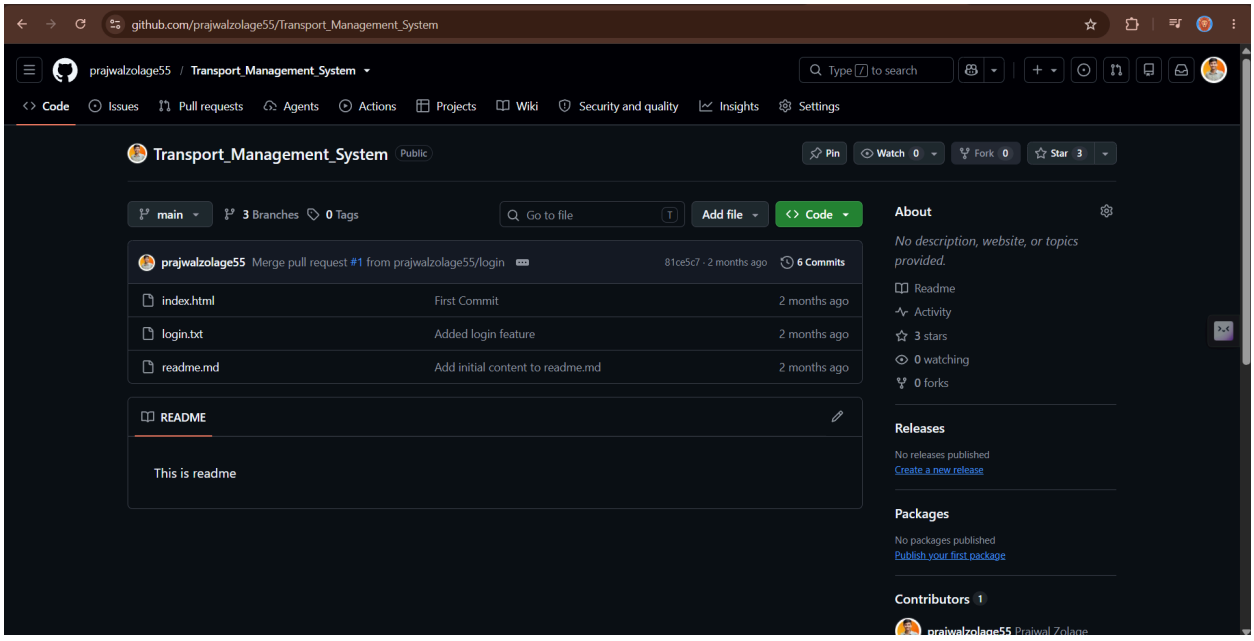
- **Pulling Latest Changes and Verifying Repository Status**

```
Output  Problems  Debug Console  Terminal  Ports                                  powershell

PS C:\Users\Admin\Transport_Management_System> git swwitch  main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
PS C:\Users\Admin\Transport_Management_System> git pull origin main
remote: Enumerating objects: 1, done.
remote: Counting objects: 100% (1/1), done.
remote: Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (1/1), 904 bytes | 180.00 KiB/s, done.
From https://github.com/shindepratham265-droid/Transport_Management_System
 * branch            main       -> FETCH_HEAD
   d8554d9..4eec415  main       -> origin/main
Updating d8554d9..4eec415
Fast-forward
 index.html | 23 ++++++++++++++++++++++
 1 file changed, 23 insertions(+)
 create mode 100644 index.html
PS C:\Users\Admin\Transport_Management_System> git log --oneline
4eec415 (HEAD -> main, origin/main, origin/HEAD) Merge pull request #1 from shindepratham265-
droid/feature-login
6e2d532 (origin/feature-login, feature-login) Added login page
d8554d9 Initial commit
PS C:\Users\Admin\Transport_Management_System> git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
PS C:\Users\Admin\Transport_Management_System>
```

Verification of the Merged Login Page



B.3 Conclusion:

Git Flow and collaborative workflows using GitHub were successfully demonstrated. A GitHub repository was created and cloned into Visual Studio Code. A feature branch was created, changes were added and committed, and the branch was pushed to GitHub. A Pull Request was created, reviewed, and merged into the main branch. Finally, the latest changes were pulled from GitHub, demonstrating collaborative development and version control.

****